GRADE 6 · CODING CLUB · LESSON 1

What is an API?

How computers ask each other for things

Every app you use is secretly talking to other computers — all day long!

TODAY

1. Apps talk to each other
2. The restaurant waiter
3. Requests and responses
4. Reading a web address
5. JSON: the language of APIs
6. Play: the Human API
7. Quiz

Apps talk to each other all the time

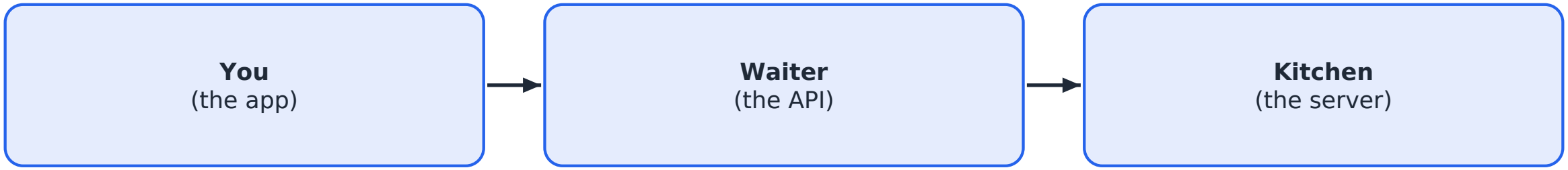
- A **weather app** asks a weather computer: "What's the weather in Auckland?"
- A **game** asks a server: "What's the high score?"
- A **map** asks: "Where is the nearest library?"
- They don't phone or text — they use an **API**
- **API** = **A**pplication **P**rogramming **I**nterface
- Simple meaning: **a set of rules for how one program asks another program for something**

TALKING POINTS

Ask: 'How does the weather app know it's going to rain? Is there a tiny weather station inside your phone?' Let them guess. Reveal: the phone asks another computer far away, using an API. Write API on the word wall.

The restaurant waiter

You don't walk into the kitchen and cook. You tell the **waiter**, and the waiter brings your food back.



1. You look at the **menu** — it lists what you're allowed to order
2. You give the waiter your **order** (the **request**)
3. The waiter brings your **food** (the **response**)
4. If you order something not on the menu, the waiter says **"Sorry, we don't have that"** (an **error**)

TALKING POINTS

Act it out with a student as the waiter. The menu is like the API's documentation — later we'll see FastAPI writes the menu for us automatically! Ask what would happen if everyone could walk into the kitchen. (Chaos, mess, not safe.) APIs keep things organised and safe.

Requests and responses

Word	Meaning	Restaurant
Request	What one program asks for	Your order
Response	What comes back	Your food
Endpoint	One thing you can ask for, at its own address	One dish on the menu
GET	"Please give me some information"	"Can I see the dessert menu?"
POST	"Please save this new information"	"Here's my birthday booking"
Error	"I can't do that" — with a reason	"Sorry, we're out of pizza"

TALKING POINTS

Keep GET and POST simple: GET = read, POST = add something new. Hand out cards with these six words and their meanings, and play a quick matching game in pairs.

Reading a web address (URL)

- `http://127.0.0.1:8000/hello/Mia?times=2`
  - `http://` — how to talk (the protocol)
  - `127.0.0.1` — which computer. 127.0.0.1 means **this** computer!
  - `:8000` — which door (the port)
  - `/hello/Mia` — the **path**: which endpoint, and a value (Mia)
  - `?times=2` — a **query**: extra options
- Like a postal address: **country → street → house → which person**

TALKING POINTS

Write the URL on the board and colour each part. Ask kids to make their own URL for a pizza API: /pizza/cheese?size=large. We'll build exactly this kind of URL in Lesson 3.

JSON: the language APIs speak

JSON looks just like a Python **dictionary**: keys and values in curly brackets

```
{
  "name": "Biscuit",
  "species": "dog",
  "age": 3,
  "is_friendly": true,
  "tricks": ["sit", "roll over", "high five"]
}
```

Spot the types:

"Biscuit" → text (string)

3 → whole number (integer)

true → yes/no (boolean)

[...] → a list

TALKING POINTS

Ask: is this a person, a pet or a car? How do you know? JSON is easy for both people and computers to read. Point out lowercase true in JSON vs True in Python — a small difference kids enjoy spotting.

Your turn: write JSON

- Write a JSON object about **yourself** (made-up details are fine!) with:
  - your **name** (text)
  - your **age** (a number — no quotes!)
  - **likes_pizza** (true or false)
  - your **favourite_foods** (a list)
- Swap with a partner: is their JSON correct? Check the brackets, quotes and commas
- Bonus: write JSON for your **favourite game character**

TALKING POINTS

Common mistakes to look for: missing commas between items, a comma after the last item, quotes around numbers, single quotes instead of double quotes. These are the same mistakes computers complain about later — a nice link to Lesson 2's validation.

Play: the Human API (15 minutes)

- Groups of 4: **App** · **API waiter** · **Server** · **Checker**
- The **menu** card lists what can be asked, e.g. `GET /joke`, `GET /animal/{name}`, `POST /pet`
- The App writes a **request card**; the waiter only accepts requests that are **on the menu**
- The Server writes the **response** as JSON on a sticky note
- The **Checker** has a **rules card** for POST /pet: name is text, age is a number from 0 to 30
  - If the rules are broken, the Checker sends back an **error** with a reason
- Swap roles every round!

TALKING POINTS

Prepare: menu cards, blank request/response cards, sticky notes, and the rules card. The Checker is secretly Pydantic (Lesson 2) and the waiter + menu is FastAPI (Lessons 3–4). Debrief: which errors happened? Was it helpful that the Checker gave a reason?

Quiz time!

- 1. What does an API do? Use the restaurant to explain.
- 2. What's the difference between a **request** and a **response**?
- 3. In `/hello/Mia?times=2`, what is the **path** and what is the **query**?
- 4. What's wrong with this JSON? `{"name": "Rex" "age": "two"}`
- **Next lesson:** we'll build the **Checker** in Python with **Pydantic**!

TALKING POINTS

Answers: 1) It lets one program ask another for something, like a waiter between you and the kitchen. 2) Request = what you ask for; response = what comes back. 3) Path: /hello/Mia; query: times=2. 4) Missing comma after "Rex", and the age should be a number (2), not text.